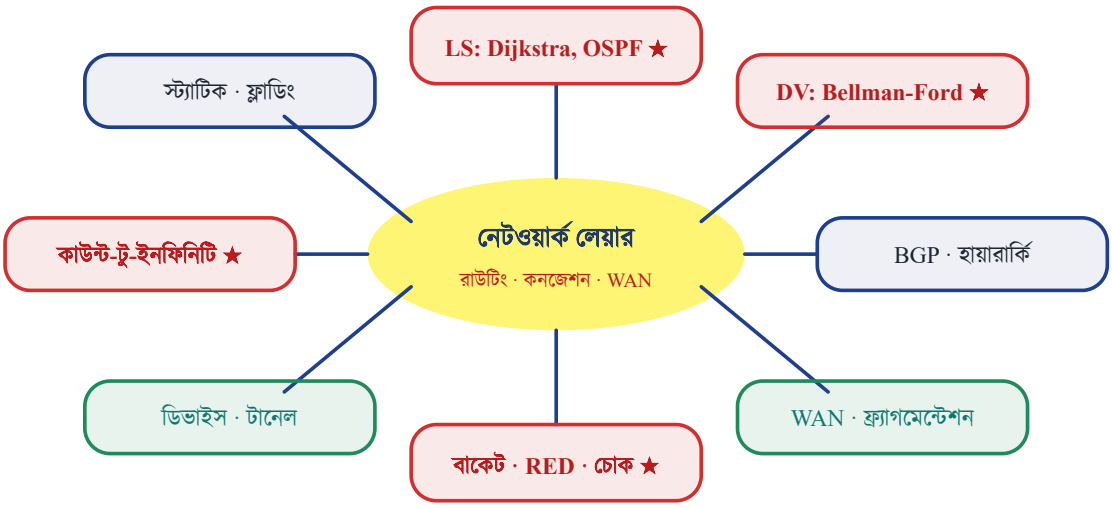
রাউটিং, কনজেশন কন্ট্রোল ও WAN

Dijkstra · Bellman-Ford · BGP · বাকেট · ডিভাইস · X.25/FR/ATM/MPLS · ফ্র্যাগমেন্টেশন: ১২ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ এক পাতায় পুরো লেসন: লাল = হাই-ইল্ড, সবুজ = প্রয়োগ

সিলেবাসের যে অংশ

"Computer Networking and Security: … **Routing algorithm**; **Congestion control**; **Internetworking, WAN**; **Fragmentation**; …"। পাশের নোটগুলোতে আছে: লেয়ার ও LAN (IEEE 802), IPv4/IPv6 হেডার ও অ্যাড্রেসিং, ARP (নেটওয়ার্ক লেয়ার নোট), TCP-র কনজেশন উইন্ডো (ট্রান্সপোর্ট লেয়ার নোট)।

হাই-ইল্ড তালিকা

① DV → Bellman-Ford
② LS → Dijkstra
③ RIP: 16 হপ = ∞
④ স্প্লিট হরাইজন
⑤ S = C/(M − ρ)
⑥ অফসেট ÷ 8 বাইট
⑦ ATM সেল 53 B

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

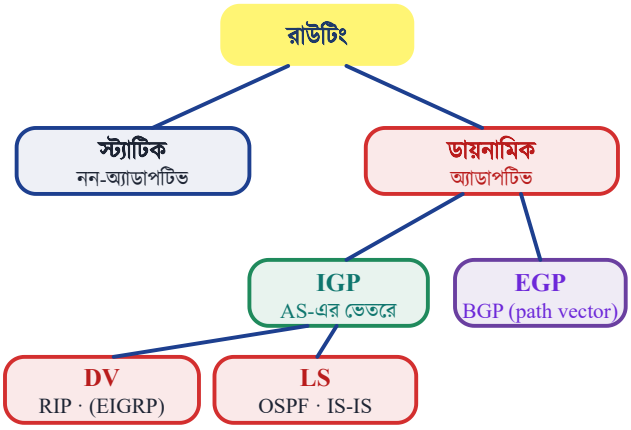
সূচিপত্র

✓ রাউটিংয়ের মূল কথা, ফ্লাডিং ..... পৃ. ২
✓ লিংক স্টেট, Dijkstra, OSPF, হায়ারার্কি … পৃ. ৩
✓ ডিসট্যান্স ভেক্টর, RIP, কাউন্ট-টু-∞ … পৃ. ৪
✓ BGP, ব্রডকাস্ট, কনজেশন ..... পৃ. ৫
✓ চোক প্যাকেট, RED, শেপিং ..... পৃ. ৬
✓ লিকি ও টোকেন বাকেট, ডিভাইস … পৃ. ৭
✓ টানেলিং, সুইচিং ..... পৃ. ৮
✓ WAN: লিজড লাইন, X.25, FR, ATM, MPLS … পৃ. ৯
✓ IPv4 ফ্র্যাগমেন্টেশন ..... পৃ. ১০
✓ পরীক্ষার আগের রাতের শিট ..... পৃ. ১১
✓ অনুশীলনী MCQ + লিখিত ..... পৃ. ১২

১ রাউটিং: পুরো চিত্র

সংজ্ঞা: **রাউটিং অ্যালগরিদম (Routing Algorithm)** হলো নেটওয়ার্ক লেয়ারের (লেয়ার ৩) সেই সফটওয়্যার, যা ঠিক করে আগত একটি প্যাকেট কোন আউটপুট লাইনে পাঠানো হবে। **রাউটিং** = রাউটিং টেবিল তৈরি করা (কন্ট্রোল প্লেন); **ফরওয়ার্ডিং (Forwarding)** = প্রতিটি প্যাকেটের জন্য সেই টেবিল দেখে প্যাকেট বের করে দেওয়া (ডেটা প্লেন)।

অপটিমালিটি প্রিন্সিপল: (Optimality Principle) I থেকে K-এর সর্বোত্তম পথে J থাকলে J থেকে K-এর সর্বোত্তম পথও সেই একই রুট ধরে যায়। তাই একটি গন্তব্যে যাওয়ার সব সর্বোত্তম রুট মিলে একটি ট্রি তৈরি করে: **সিংক ট্রি (Sink Tree)** (কোনো লুপ নেই)।



রাউটিংয়ের বংশতালিকা

	স্ট্যাটিক (নন-অ্যাডাপটিভ)	ডায়নামিক (অ্যাডাপটিভ)
রুট ঠিক করে	অ্যাডমিনিস্ট্রেটর, হাতে	রাউটারগুলোর প্রোটোকল-বিনিময়
বিকল হলে · ওভারহেড · ব্যবহার	সাড়া দেয় না · নেই · ছোট/স্টাব নেটওয়ার্ক	নিজেই রুট বদলায় · ব্যান্ডউইথ, CPU · বড় নেটওয়ার্ক

মেট্রিক (Metric): হপ কাউন্ট (RIP) · ব্যান্ডউইথ · ডিলে · লোড · নির্ভরযোগ্যতা · MTU · কস্ট (OSPF) · AS পাথ / পলিসি (BGP)। **মেট্রিক যত কম, রুট তত ভালো**।

মনে রাখার কৌশল: **"RIP গোনে হপ, OSPF গোনে কস্ট, BGP গোনে AS** (আর মানে পলিসি)"।

পরীক্ষার ফাঁদ: **মেট্রিক** তুলনা করে **একই প্রোটোকলের ভেতরের** রুটগুলো; **অ্যাডমিনিস্ট্রেটিভ ডিসট্যান্স** (AD, বিশ্বাসযোগ্যতা) বাছাই করে **ভিন্ন প্রোটোকলের মধ্যে**, যার মান কম সে জেতে। Cisco AD: connected 0 · static **1** · eBGP 20 · EIGRP 90 · OSPF 110 · IS-IS 115 · RIP 120 · iBGP 200। তাই হপ কাউন্ট যা-ই হোক, OSPF রুট RIP রুটকে হারায়।

২ ফ্লাডিং (Flooding)

একটি স্ট্যাটিক অ্যালগরিদম: প্রতিটি প্যাকেট **যে লাইনে এসেছে সেটি ছাড়া বাকি সব লাইনে** পাঠানো হয়। এটি মজবুত (robust) এবং সবসময় সংক্ষিপ্ততম পথ খুঁজে পায় (সব পথই চেষ্টা হয়), কিন্তু নেটওয়ার্ক ডুপ্লিকেট প্যাকেটে ভরে যায়। নিয়ন্ত্রণের উপায়: **হপ কাউন্টার** (শুরু হয় নেটওয়ার্কের ব্যাস থেকে), **সিকোয়েন্স নম্বর** (আগে দেখা কপি বাদ) বা **সিলেক্টিভ ফ্লাডিং**। লিংক-স্টেট প্যাকেট ছড়াতে ফ্লাডিং ব্যবহৃত হয়।

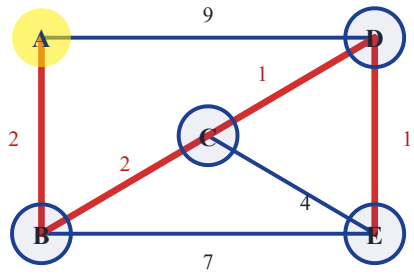
৩ লিংক স্টেট রাউটিং ও Dijkstra

মূল ধারণা: প্রতিটি রাউটার **পুরো টপোলজির মানচিত্র** জানে এবং নিজেই তার ওপর **Dijkstra অ্যালগরিদম** (SPF) চালায়: "মানচিত্র দেখে রাউটিং"।

১. **প্রতিবেশী খোঁজা** (HELLO) · প্রত্যেকের **কস্ট মাপা** · **লিংক স্টেট প্যাকেট তৈরি** (LSP/LSA: ID, সিকোয়েন্স নং, বয়স, প্রতিবেশী + কস্ট)

২. LSP **সব** রাউটারে **ফ্লাড** করা · পুরো মানচিত্রে **Dijkstra চালানো** → সংক্ষিপ্ততম-পথ ট্রি → রাউটিং টেবিল

সমাধানকৃত উদাহরণ: A থেকে সংক্ষিপ্ততম পথ



লাল = চূড়ান্ত সংক্ষিপ্ততম-পথ ট্রি

ধাপ	স্থায়ী সেট	B	C	D	E
0	{A}	**2,A**	∞	9,A	∞
1	{A,B}	✓	**4,B**	9,A	9,B
2	{A,B,C}	✓	✓	**5,C**	8,C
3	{A,B,C,D}	✓	✓	✓	**6,D**

"4,B" = দূরত্ব 4, আগের নোড B। প্রতি ধাপে **সবচেয়ে ছোট অস্থায়ী (tentative)** নোডটি স্থায়ী হয় এবং তার প্রতিবেশীদের মান হালনাগাদ (relax) হয়। A থেকে সব রুটের নেক্সট হপ **B**; A→E = A-B-C-D-E, কস্ট 6 (সরাসরি মনে হওয়া A-D-E = 10 নয়)।

OSPF-এর তথ্য:

RFC 2328 (v2), IPv6-এর জন্য v3
সরাসরি IP-র ওপর চলে: প্রোটোকল **89**
কস্ট = 10^8 / ব্যান্ডউইথ (100 Mbps → 1)
এরিয়া (Area); ব্যাকবোন = **area 0**; ABR, ASBR, DR/BDR
224.0.0.5 সব রাউটার · 224.0.0.6 DR
Hello 10 s, dead 40 s · AD 110

মনে রাখার কৌশল: সংখ্যায় OSPF: "**89 · 110 · area 0 · 5/6 · 10/40**" = প্রোটোকল, AD, ব্যাকবোন, মাল্টিকাস্টের শেষ অঙ্ক, hello/dead।

IS-IS: OSI-র লিংক-স্টেট যমজ, IP নয়, সরাসরি লেয়ার ২-এর ওপর চলে। Dijkstra: $O(N^2)$।

পরীক্ষার ফাঁদ: **ঋণাত্মক** এজ কস্টে Dijkstra ভুল ফল দেয়; Bellman-Ford তা সামলায় (তবে ঋণাত্মক সাইকেল নয়)।

৪ হায়ারার্কিক্যাল রাউটিং (Hierarchical Routing)

রাউটারগুলোকে **অঞ্চলে (region)** ভাগ করা হয় (এরিয়া, AS); একটি রাউটার নিজের অঞ্চলের সব রাউটার জানে, কিন্তু অন্য প্রতিটি অঞ্চলের জন্য রাখে **মাত্র একটি এন্ট্রি**। টেবিল ছোট হয়, পথ সামান্য লম্বা হতে পারে। সর্বোত্তম: **ln N** স্তর, **e·ln N** এন্ট্রি।

সমাধানকৃত উদাহরণ ✍️: 720টি রাউটার। ফ্ল্যাট: **720** এন্ট্রি। 24 অঞ্চল × 30 রাউটার: 30 + 23 = **53**। 8 ক্লাস্টার × 9 অঞ্চল × 10 রাউটার: 10 + 8 + 7 = **25**।

৫ ডিসট্যান্স ভেক্টর রাউটিং ও Bellman-Ford

মূল ধারণা: প্রতিটি রাউটার প্রতিটি গন্তব্যের সবচেয়ে ভালো জানা **দূরত্ব** (ও লাইন) রাখে এবং এই ভেক্টর পর্যায়ক্রমে **শুধু প্রতিবেশীদের সাথে** বিনিময় করে। "শোনা কথায় রাউটিং"; ১৯৭৯ পর্যন্ত ARPANET-এ, পরে RIP-এ।

Bellman-Ford সমীকরণ: $D_x(y) = \min_v \{ c(x, v) + D_v(y) \}$ x-এর সব প্রতিবেশী v-এর ওপর

সমাধানকৃত উদাহরণ ✍️: রাউটার X: লিংক X–A = 2, X–B = 5, X–C = 1। প্রাপ্ত ভেক্টর:

কার কাছ থেকে	Y পর্যন্ত	Z পর্যন্ত
A	7	3
B	2	9
C	8	6

Y: min(2+7, **5+2**, 1+8) = **7, B হয়ে**

Z: min(**2+3**, 5+9, 1+6) = **5, A হয়ে**

RIP-এর তথ্য:

মেট্রিক = **হপ কাউন্ট**; সর্বোচ্চ **15**, **16** = ∞

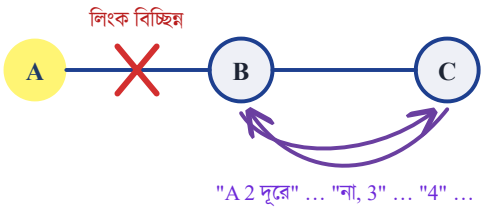
প্রতি **30 s**-এ পুরো টেবিল · UDP পোর্ট **520**

invalid 180 s · hold-down 180 s · flush 240 s

v1 ক্লাসফুল, ব্রডকাস্ট · v2 ক্লাসলেস, মাল্টিকাস্ট **224.0.0.9**, অথেনটিকেশন

RIPng (IPv6): UDP 521 · AD 120

৬ কাউন্ট-টু-ইনফিনিটি ও তার প্রতিকার



বিনিময়	B থেকে A-র দূরত্ব	C থেকে A-র দূরত্ব
বিকল হওয়ার আগে	1	2
1	3 (C হয়ে!)	2
2 · 3 · 4	3 · 5 · 5	4 · 4 · 6
শেষে	**16 = ∞**	**16 = ∞**

C তখনও "A 2 দূরে" ঘোষণা করে (যে রুট আসলে B-এর মধ্য দিয়েই যায়), আর B তা বিশ্বাস করে। **সুখবর দ্রুত ছড়ায়, দুঃসংবাদ ধীরে।**

প্রতিকার	কীভাবে কাজ করে
স্প্লিট হরাইজন · পয়জন রিভার্স	যে ইন্টারফেসে রুট শেখা, সেখানে তা আর ফেরত ঘোষণা নয় · অথবা ফেরত পাঠাও মেট্রিক 16 (∞) দিয়ে
রুট পয়জনিং · ট্রিগারড আপডেট	বিকল রুট 30 s অপেক্ষা না করে সঙ্গে সঙ্গে মেট্রিক 16 দিয়ে পাঠানো হয়
হোল্ড-ডাউন টাইমার	রুট ডাউন হলে 180 s তার সম্পর্কে খারাপ খবর উপেক্ষা

পরীক্ষার ফাঁদ: স্প্লিট হরাইজন দুই রাউটারের লুপ সারায়, কিন্তু তিন বা তার বেশি রাউটারের লুপ **নয়**; RIP-এর ছোট ∞ (16) গণনাকে শুধু সীমিত করে।

	ডিসট্যান্স ভেক্টর	লিংক স্টেট
জানে · অ্যালগরিদম	প্রতিবেশীর দূরত্ব · **Bellman-Ford**	পুরো টপোলজি · **Dijkstra**
পাঠায়	পুরো টেবিল, প্রতিবেশীকে, পর্যায়ক্রমে	নিজের লিংক (LSA), সবাইকে, পরিবর্তন হলে
কনভারজেন্স · CPU	ধীর, কাউন্ট-টু-∞ · কম, ছোট নেটওয়ার্ক	দ্রুত, লুপমুক্ত · বেশি, বড় নেটওয়ার্ক
উদাহরণ	RIP, IGRP; EIGRP = "অ্যাডভান্সড DV / হাইব্রিড" (DUAL)	OSPF, IS-IS

৭ BGP: অটোনোমাস সিস্টেমের মধ্যে রাউটিং

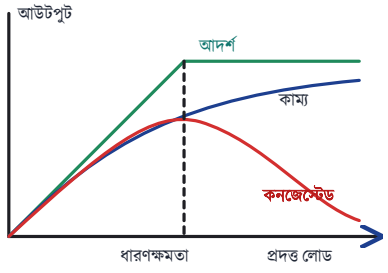
- **পাথ ভেক্টর (Path Vector)** প্রোটোকল; BGP-4 হলো ইন্টারনেটের ইন্টার-ডোমেইন (EGP) প্রোটোকল
- চলে **TCP পোর্ট 179**-এ; AS-এর মধ্যে eBGP, একই AS-এর ভেতরে iBGP
- প্রতিটি রুট তার পুরো **AS_PATH** বহন করে; যে পথে নিজের AS আছে রাউটার তা বাতিল করে → **লুপ হয় না**
- বাছাই করে **পলিসি** দিয়ে (LOCAL_PREF, AS_PATH-এর দৈর্ঘ্য, MED), কস্ট দিয়ে নয় · মেসেজ: OPEN, UPDATE, KEEPALIVE, NOTIFICATION

৮ ব্রডকাস্ট ও মাল্টিকাস্ট রাউটিং

ব্রডকাস্ট: আলাদা আলাদা কপি (অপচয়) · ফ্লাডিং · মাল্টিডেস্টিনেশন · **স্প্যানিং ট্রি** (সর্বোত্তম) · **রিভার্স পাথ ফরওয়ার্ডিং** (RPF): কপিটি যদি সেই লাইনে আসে যা দিয়ে এই রাউটার নিজে উৎসে পৌঁছায়, তবেই ফরওয়ার্ড করো। **মাল্টিকাস্ট** (224.0.0.0/4, IGMP): সোর্স-ভিত্তিক ট্রি (DVMRP, PIM-DM) অথবা শেয়ার্ড কোর-ভিত্তিক ট্রি (CBT, PIM-SM)।

৯ কনজেশন ও তার নিয়ন্ত্রণ

সংজ্ঞা: **কনজেশন (Congestion)**: নেটওয়ার্কে (বা তার কোনো অংশে) অতিরিক্ত প্যাকেট, ফলে ডিলে বাড়ে ও থ্রুপুট কমে। **কনজেশন কন্ট্রোল** গোটা নেটওয়ার্কের বিষয় (global); **ফ্লো কন্ট্রোল** পয়েন্ট-টু-পয়েন্ট (দ্রুত প্রেরক, ধীর প্রাপক)।



প্রদত্ত বনাম পৌঁছানো প্যাকেট

কারণ: একাধিক লাইনের বার্স্টি ট্রাফিক এক লাইনে জড়ো হওয়া, ধীর রাউটার, কম বাফার, কম ব্যান্ডউইথের লাইন।

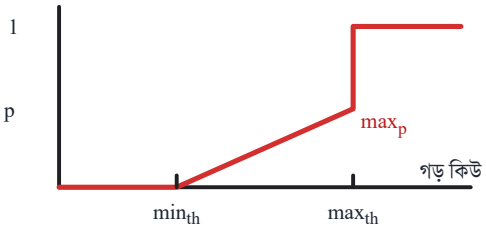
অসীম বাফার অবস্থা আরও **খারাপ** করে: প্যাকেট এত দেরি করে যে টাইমআউটে রিট্রান্সমিট হয়, লোড আরও বাড়ে। ধারণক্ষমতা পেরোলে আউটপুট প্রায় শূন্যে নামতে পারে: **কনজেশন কলাপস**।

	ওপেন লুপ (প্রতিরোধ)	ক্লোজড লুপ (ফিডব্যাক, প্রতিক্রিয়া)
ধারণা	ভালো ডিজাইন, যাতে কনজেশন শুরুই না হয়	পর্যবেক্ষণ, সংকেত, তারপর সমন্বয়
উপায়	প্রভিশনিং, **অ্যাডমিশন কন্ট্রোল**, ট্রাফিক শেপিং (বাকেট), রিট্রান্সমিশন/ACK/ডিসকার্ড পলিসি	**চোক প্যাকেট**, ব্যাক-প্রেশার, ECN, RED, TCP-র AIMD উইন্ডো (ট্রান্সপোর্ট নোট)

- **অ্যাডমিশন কন্ট্রোল**: ভার্চুয়াল-সার্কিট নেটওয়ার্কে নতুন VC নেটওয়ার্ককে কনজেস্টেড করলে তা প্রত্যাখ্যান (টেলিফোনের "ব্যস্ত" টোনের মতো)।

- **চোক প্যাকেট (Choke Packet)**: কনজেস্টেড রাউটার **উৎসে ফেরত** একটি প্যাকেট পাঠায়, উৎস তার রেট কমায় (যেমন অর্ধেক)। **হপ-বাই-হপ ব্যাক-প্রেশার**: চোক প্যাকেট পথের প্রতিটি রাউটারেই কার্যকর হয়, তাই তাৎক্ষণিক স্বস্তি। (ICMP Source Quench এখন বাতিল।)
- **ECN**: রাউটার প্যাকেট ড্রপ না করে IP হেডারের 2 বিট **মার্ক** করে; প্রাপক তা ফেরত জানায় (TCP ECE ফ্ল্যাগ), প্রেরক গতি কমায়।
- **লোড শেডিং**: প্যাকেট ফেলে দেওয়া। **ওয়াইন** (পুরোনোটাই ভালো: ফাইল ট্রান্সফার, নতুনটা ফেলো) বনাম **মিল্ক** (নতুনটাই ভালো: রিয়েল-টাইম মিডিয়া, পুরোনোটা ফেলো)।

RED (Random Early Detection, Floyd ও Jacobson ১৯৯৩): রাউটার কিউয়ের **গড়** দৈর্ঘ্য দেখে। $\min_{th}$-এর নিচে: ড্রপ নেই; দুই থ্রেশহোল্ডের মাঝে: $\max_p$ পর্যন্ত বাড়তে থাকা সম্ভাবনায় ড্রপ; $\max_{th}$-এর ওপরে: সব ড্রপ। **আগেভাগে ও এলোমেলোভাবে** ড্রপ করায় অল্প কয়েকটি TCP প্রেরক ধীর হয়, টেইল-ড্রপের **গ্লোবাল সিনক্রোনাইজেশন** এড়ানো যায়।

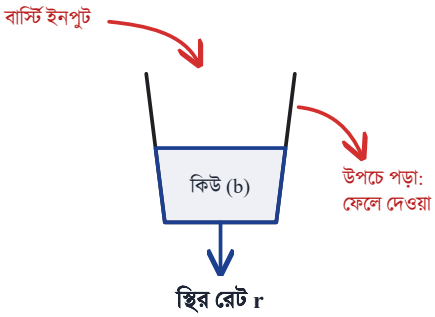


RED-এর ড্রপ সম্ভাবনা

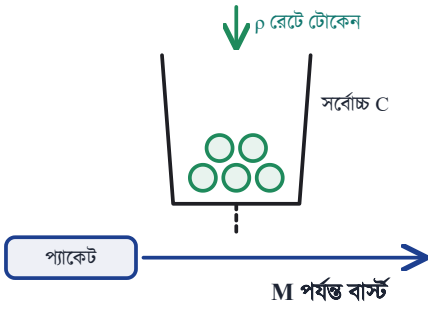
পরীক্ষার ফাঁদ: কনজেশন কন্ট্রোল ≠ ফ্লো কন্ট্রোল: ফাঁকা লিংকে দ্রুত PC থেকে ধীর প্রিন্টারে পাঠানো ফ্লো-কন্ট্রোলের সমস্যা; এক ওভারলোডেড লিংকে বহু হোস্ট হলো কনজেশন।

১০ ট্রাফিক শেপিং: লিকি ও টোকেন বাকেট

ট্রাফিক শেপিং (Traffic Shaping): হোস্ট যে ট্রাফিক পাঠায় তার **রেট ও বার্স্টিনেস** নেটওয়ার্কের সাথে চুক্তি অনুযায়ী নিয়ন্ত্রণ (ওপেন লুপ)। **পুলিশিং (Policing)** = নেটওয়ার্ক তা যাচাই করে অতিরিক্ত অংশ ড্রপ বা মার্ক করে।



লিকি বাকেট: মসৃণ আউটপুট



টোকেন বাকেট: বার্স্ট চলে

	লিকি বাকেট	টোকেন বাকেট
আউটপুট	ইনপুট যত বার্স্টিই হোক, **স্থির** রেট r	গড় রেট ρ, M পর্যন্ত বার্স্ট চলে
অলস হোস্ট · পূর্ণ হলে	কিছু পায় না · **প্যাকেট** ফেলে দেয়	C পর্যন্ত টোকেন জমায় · **টোকেন** ফেলে দেয়
t সময়ে সর্বোচ্চ ডেটা	r·t	**C + ρt** (এবং ≤ M·t)

বার্স্টের দৈর্ঘ্য: পূর্ণ বাকেট C বাইট, টোকেন রেট ρ, সর্বোচ্চ আউটপুট রেট M। S সেকেন্ডের বার্স্টে: $C + \rho S = MS \Rightarrow \mathbf{S = C / (M - \rho)}$

উদাহরণ ১ ✍️: C = 250 KB, M = 25 MB/s, ρ = 2 MB/s: S = 250 KB / 23 MB/s ≈ **11 ms** পূর্ণ গতিতে, তারপর 2 MB/s।

উদাহরণ ২ ✍️: 1 MB বার্স্ট (25 MB/s-এ 40 ms) লিকি বাকেটে, আউটপুট 2 MB/s: আউটপুট চলে **500 ms**; বাকেটে ধরতে হবে ≈ 0.92 MB।

মনে রাখার কৌশল: **লিকি** = ফুটো কল: সবসময় একই ফোঁটা। **টোকেন** = সঞ্চয়ী হিসাব: অলস সময়ে জমাও, বার্স্টে খরচ করো, তবে ব্যালান্স C-এর বেশি কখনো নয়।

১১ ইন্টারনেটওয়ার্কিং ও তার ডিভাইস

সংজ্ঞা: **ইন্টারনেটওয়ার্কিং (Internetworking)** = ভিন্ন ধরনের নেটওয়ার্ক (অ্যাড্রেসিং, প্যাকেটের আকার, সার্ভিস, QoS) জুড়ে এক নেটওয়ার্কের মতো চালানো; IP চালানো লেয়ার-৩ রাউটারই ইন্টারনেটকে জুড়ে রাখে।

ডিভাইস	লেয়ার	কাজ করে	কলিশন ডোমেইন	ব্রডকাস্ট ডোমেইন
রিপিটার	1	বিট: সিগন্যাল পুনর্গঠন	1 (বাড়ায়)	1
হাব	1	বিট: মাল্টিপোর্ট রিপিটার	সব পোর্টে মিলে 1	1
ব্রিজ	2	ফ্রেম: MAC ফিল্টারিং	প্রতি পোর্টে 1	1
সুইচ	2	ফ্রেম: মাল্টিপোর্ট ব্রিজ	**প্রতি পোর্টে 1**	1 (প্রতি VLAN-এ)
রাউটার	3	প্যাকেট: IP অ্যাড্রেস	প্রতি পোর্টে 1	**প্রতি পোর্টে 1**
গেটওয়ে	4–7	প্রোটোকল রূপান্তর	–	–

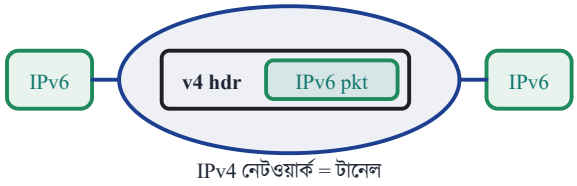
- **ব্রিজ**: সোর্স অ্যাড্রেস দেখে MAC → পোর্ট শেখে (**ব্যাকওয়ার্ড লার্নিং**); অজানা গন্তব্যে ফ্লাড করে; STP (802.1D) লুপ দূর করে।
- **সুইচ**: store-and-forward (CRC যাচাই করে) বা cut-through (কম ল্যাটেন্সি); ফুল ডুপ্লেক্স, কলিশন নেই; লেয়ার-৩ সুইচ রাউটিংও করে।
- **গেটওয়ে**: ভিন্ন প্রোটোকলের মধ্যে রূপান্তর করে (ই-মেইল, VoIP); TCP/IP-র "ডিফল্ট গেটওয়ে" আসলে রাউটারই।

সমাধানকৃত উদাহরণ ✍️: ২টি ইন্টারফেসের একটি রাউটার: একটি ইন্টারফেসে ২০টি PC-সহ ২৪-পোর্ট সুইচ, অন্যটিতে ৬টি PC-সহ একটি হাব। কলিশন ডোমেইন: 20 + 1 (সুইচ–রাউটার লিংক) + 1 (পুরো হাব অংশ) = **22**। ব্রডকাস্ট ডোমেইন: **2** (রাউটারের প্রতি ইন্টারফেসে একটি)।

পরীক্ষার ফাঁদ: **সুইচ** কলিশন ডোমেইন ভাগ করে, কিন্তু ব্রডকাস্ট ডোমেইন **নয়**; ব্রডকাস্ট ডোমেইন ভাগ করে কেবল **রাউটার** (বা VLAN)। অনেক পোর্ট থাকলেও হাব লেয়ার-১ ডিভাইস।

টানেলিং (Tunnelling)

ভিন্ন ধরনের নেটওয়ার্ক পার হতে দুটি মাল্টিপ্রোটোকল রাউটারের মাঝে পুরো প্যাকেটটি ওই নেটওয়ার্কের প্যাকেটের **পেলোড** হিসেবে যায়। উদাহরণ: IPv6-in-IPv4 (প্রোটোকল 41), GRE (প্রোটোকল 47), IPsec VPN।



১২ সার্কিট বনাম প্যাকেট সুইচিং

	সার্কিট	ডেটাগ্রাম	ভার্চুয়াল সার্কিট
ধারণা	আগে নির্দিষ্ট পথ স্থাপন	প্রতিটি প্যাকেট আলাদাভাবে রাউট হয়	সেটআপে পথ নির্দিষ্ট, লিংক শেয়ার্ড
প্যাকেটে থাকে	–	পূর্ণ অ্যাড্রেস	ছোট VC নম্বর
ক্রম · রাউটার বিকল হলে	ক্রমানুসারে · কল বিচ্ছিন্ন	ক্রম ভাঙতে পারে · শুধু তার প্যাকেট হারায়	ক্রমানুসারে · তার মধ্য দিয়ে যাওয়া সব VC শেষ
QoS, কনজেশন	সহজ (রিজার্ভড)	কঠিন	রিসোর্স রিজার্ভ থাকলে সহজ
উদাহরণ	PSTN, ISDN	**IP**	**X.25, Frame Relay, ATM, MPLS**

ডিলের সূত্র: মেসেজ x বিট, k হপ, বিট রেট b, প্রতি হপে প্রোপাগেশন d, সেটআপ s, প্যাকেট p বিট (হেডার বাদ): সার্কিট $\mathbf{T = s + x/b + k\cdot d}$ · প্যাকেট $\mathbf{T = x/b + (k-1)\cdot p/b + k\cdot d}$। $s > (k-1)\cdot p/b$ হলে প্যাকেট সুইচিং দ্রুত।
উদাহরণ ✍️ $x = 10^6$ বিট, $b = 1$ Mbps, $k = 4$, $d = 5$ ms, $s = 200$ ms, $p = 10\,000$ বিট: সার্কিট $0.2 + 1 + 0.02 = \mathbf{1.22\ s}$; প্যাকেট $1 + 3 \times 0.01 + 0.02 = \mathbf{1.05\ s}$।

পরীক্ষার ফাঁদ: ভার্চুয়াল সার্কিট আসলে **প্যাকেট** সুইচিং (লিংক শেয়ার হয়), সার্কিট সুইচিং নয়; এটি শুধু সেটআপের সময় রুট নির্দিষ্ট করে।

১৩ WAN প্রযুক্তি

WAN শহর, দেশ, মহাদেশ জুড়ে বিস্তৃত, সাধারণত ক্যারিয়ারের লাইনের ওপর: লিজড লাইন, সার্কিট-সুইচড ISDN, প্যাকেট-সুইচড X.25 / Frame Relay / ATM, MPLS, ইন্টারনেট VPN (IPsec), স্যাটেলাইট/সেলুলার ব্যাকআপ।

লিজড লাইন	রেট	চ্যানেল
T1 (USA)	**1.544 Mbps**	24 × 64 k
E1 (ইউরোপ, BD)	**2.048 Mbps**	32 (30 ব্যবহারযোগ্য)
T3 / E3	44.736 / 34.368	672 / 480
OC-3 = STM-1	155.52 Mbps	SONET/SDH
OC-12 = STM-4	622.08 Mbps	

ISDN: BRI = 2B + D = 2 × 64 + 16 = **144 kbps**; PRI = 23B + D (T1) বা 30B + D (E1)।

T1 = 24 × 64 kbps + 8 kbps ফ্রেমিং = 1.544 Mbps। OC-1 = 51.84 Mbps; OC-n = n × 51.84।

	X.25	Frame Relay	ATM
সময় · সংস্থা	1976, CCITT/ITU-T	1990-এর দশকের শুরু	1990-এর দশক, B-ISDN-এর মূল
লেয়ার	3 (X.21, LAPB, PLP)	শুধু 2 (LAPF)	ফিজিক্যাল, ATM, AAL
একক	প্যাকেট (পরিবর্তনশীল)	ফ্রেম (পরিবর্তনশীল)	**53-বাইট সেল**
এরর কন্ট্রোল	**হপ-বাই-হপ** (নয়েজি লাইন)	নেই: ত্রুটিপূর্ণ ফ্রেম ড্রপ	HEC শুধু হেডারে
VC শনাক্তকারী	LCN	**DLCI** (10 বিট)	**VPI/VCI**
গতি	≤ 64 kbps (পরে 2 Mbps)	56 kbps – 45 Mbps	155.52 / 622.08 Mbps
অতিরিক্ত	টার্মিনালের জন্য PAD	CIR, DE বিট, FECN/BECN	CBR, VBR, ABR, UBR



ATM সেল



MPLS লেবেল এন্ট্রি

ATM: 48 বাইট ছিল আপস (USA চেয়েছিল 64, ইউরোপ 32)। UNI হেডার: GFC 4, VPI 8, VCI 16, PT 3, CLP 1, HEC 8 বিট। ATM নেটওয়ার্ক লেয়ার ও AAL আলাদা নোটে।

MPLS ("লেয়ার 2.5", RFC 3031): ইনগ্রেস (প্রবেশপ্রান্তের) **LER** প্যাকেটকে একটি FEC-এ শ্রেণিভুক্ত করে লেবেল **push** করে; কোরের **LSR** লেবেল **swap** করে (হুবহু মিল, দ্রুত); ইগ্রেস (নির্গমপ্রান্তে) লেবেল **pop** হয়। পথটির নাম **LSP**। ক্যারিয়াররা ট্রাফিক ইঞ্জিনিয়ারিং, QoS ও L3 VPN-এ ব্যবহার করে; যেকোনো প্রোটোকল বহন করে।

মনে রাখার কৌশল: **X.25 প্রতি হপে যাচাই করে, Frame Relay কোথাও করে না, ATM সব কেটে 53 বানায়।** Frame Relay = "এরর কন্ট্রোল ফেলে দেওয়া X.25, কারণ ফাইবার পরিষ্কার"।

পরীক্ষার ফাঁদ: ATM-এর HEC শুধু **5-বাইট হেডার** রক্ষা করে, পেলোড নয়। Frame Relay ত্রুটি **শনাক্ত** করে (FCS) কিন্তু কখনো **সংশোধন** করে না: ফ্রেমটি ফেলে দেয়।

১৪ IPv4 ফ্র্যাগমেন্টেশন (Fragmentation)

কেন: পরের লিংকের **MTU**-র (একটি ফ্রেমের সর্বোচ্চ পেলোড) চেয়ে বড় ডেটাগ্রাম কয়েকটি **ফ্র্যাগমেন্টে** ভাগ হয়, প্রতিটি নিজস্ব হেডারসহ পূর্ণ IP ডেটাগ্রাম। IPv4: উৎস **বা যেকোনো রাউটার** ফ্র্যাগমেন্ট করতে পারে; **রিঅ্যাসেম্বলি হয় শুধু চূড়ান্ত গন্তব্যে**।

হেডার ফিল্ড (বিট)	ভূমিকা
Identification (16)	সব ফ্র্যাগমেন্টে একই
DF (1) · MF (1)	ফ্র্যাগমেন্ট নয় · আরও আছে
Fragment offset (**13**)	**8-বাইট** এককে
Total length (16)	হেডার + ডেটা ≤ 65 535

লিংক	MTU (B)
Ethernet · PPP	**1500**
802.11 · FDDI	2304 · 4352
X.25 (সাধারণত)	576
IPv6 ন্যূনতম	1280

সমাধানকৃত উদাহরণ ১ ✍️ : টোটাল লেংথ 4000 B (20-বাইট হেডার), পরের লিংকের MTU 1500।

১ ডেটা = 4000 − 20 = 3980 B। প্রতি ফ্র্যাগমেন্টে সর্বোচ্চ ডেটা = 1500 − 20 = 1480 (8-এর গুণিতক ✓)

২ ভাগ: 3980 = 1480 + 1480 + 1020

ফ্র্যাগ	ডেটা বাইট	টোটাল লেংথ	অফসেট	MF
1	0 – 1479	1500	0	1
2	1480 – 2959	1500	1480/8 = 185	1
3	2960 – 3979	1040	**2960/8 = 370**	**0**

উদাহরণ ২: 8-বাইটের ফাঁদ ✍️ : MTU 1000, ডেটাগ্রাম 2400 B। 1000 − 20 = 980, যা 8-এর গুণিতক নয় → নিতে হবে **976**। ডেটা 2380 = 976 + 976 + 428; অফসেট **0, 122, 244**।

উদাহরণ ৩: ফ্র্যাগমেন্ট পড়া ✍️ : অফসেট 100, HLEN 5, টোটাল লেংথ 100 → প্রথম ডেটা বাইট = 100 × 8 = **800**, ডেটা = 80 B, শেষ বাইট **879**।

- MF = 1, অফসেট = 0 → **প্রথম** ফ্র্যাগমেন্ট · MF = 0, অফসেট ≠ 0 → **শেষ** · MF = 0, অফসেট = 0 → ফ্র্যাগমেন্ট হয়নি
- একটি ফ্র্যাগমেন্ট হারালে **পুরো ডেটাগ্রাম** বাতিল হয় (রিঅ্যাসেম্বলি টাইমার)
- **পাথ MTU ডিসকভারি**: DF = 1 দিয়ে পাঠাও; যে রাউটার ফরওয়ার্ড করতে পারে না সে প্যাকেট ড্রপ করে ICMP type 3 code 4 "fragmentation needed" ও পরের হপের MTU ফেরত পাঠায়
- **IPv6**: রাউটার **কখনো** ফ্র্যাগমেন্ট করে না; শুধু উৎস করে (Fragment হেডার, 8 B, next header 44); বেশি বড় হলে → ICMPv6 "Packet Too Big"

মনে রাখার কৌশল: "**অফসেট × 8 = বাইটের অবস্থান**": 2^{13}টি অফসেট × 8 বাইট = 2^{16}, যা 65 535-বাইটের ডেটাগ্রাম ঢেকে দেয়।

পরীক্ষার আগের রাতের শিট

তথ্য-তালিকা:

DV: $D_x(y) = \min\{c(x,v) + D_v(y)\}$

LS: LSA ফ্লাড, তারপর Dijkstra $O(N^2)$

RIP: হপ, সর্বোচ্চ 15, 16 = ∞, 30 s, UDP 520

OSPF: IP প্রোটোকল 89, area 0, AD 110

BGP: পাথ ভেক্টর, TCP 179, AS_PATH

AD: static 1, eBGP 20, OSPF 110, RIP 120

হায়ারার্কি: ln N স্তর, e·ln N এন্ট্রি

টোকেন: S = C/(M − ρ) · সর্বোচ্চ = C + ρt

সার্কিট: s + x/b + kd

প্যাকেট: x/b + (k − 1)p/b + kd

ATM 53 = 5 + 48 · VPI/VCI

T1 1.544 · E1 2.048 · STM-1 155.52

MPLS: label 20 · TC 3 · S 1 · TTL 8

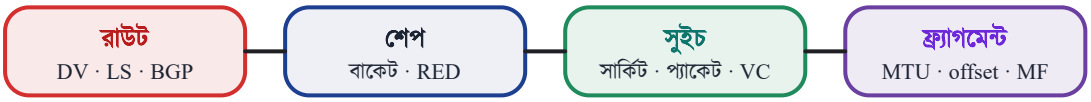
অফসেট = বাইটের অবস্থান / 8 (13 বিট)

MCQ-র জন্য সেরা ১৪টি এক-লাইনের তথ্য

1. রাউটার = লেয়ার ৩, ব্রিজ/সুইচ = ২, হাব/রিপিটার = ১
2. ডিসট্যান্স ভেক্টর Bellman-Ford ব্যবহার করে
3. লিংক স্টেট Dijkstra ব্যবহার করে
4. BGP হলো AS-এর মধ্যকার (EGP) প্রোটোকল
5. স্প্লিট হরাইজন কাউন্ট-টু-ইনফিনিটি কমায়
6. এক গন্তব্যের সব সর্বোত্তম রুট = সিংক ট্রি
7. ফ্লাডিং সব পথ চেষ্টা করে: মজবুত, প্রচুর ডুপ্লিকেট
8. লিকি বাকেট → স্থির আউটপুট রেট
9. টোকেন বাকেট → বার্স্ট চলে, টোকেন জমায়
10. RED গড় কিউ দেখে আগেভাগে, এলোমেলো ড্রপ করে
11. X.25 = হপ-বাই-হপ এরর কন্ট্রোল
12. Frame Relay-তে DLCI; ATM-এ VPI/VCI
13. IPv4 রিঅ্যাসেম্বলি শুধু গন্তব্যে
14. IPv6 রাউটার কখনো ফ্র্যাগমেন্ট করে না

প্রধান ফাঁদ: মেট্রিক ≠ অ্যাডমিনিস্ট্রেটিভ ডিসট্যান্স · ৩+ রাউটারের লুপে স্প্লিট হরাইজন ব্যর্থ · সুইচ কলিশন ডোমেইন ভাগ করে, ব্রডকাস্ট নয় · ভার্চুয়াল সার্কিট = প্যাকেট সুইচিং · লিকি বাকেট প্যাকেট ফেলে, টোকেন বাকেট টোকেন ফেলে · অফসেট গোনে 8-বাইট এককে, প্রতি ফ্র্যাগমেন্টের ডেটা 8-এর গুণিতক হতে হবে · ATM-এর HEC শুধু হেডারে

সব কৌশল একসাথে: RIP গোনে হপ, OSPF কস্ট, BGP AS · OSPF "89 · 110 · area 0 · 5/6 · 10/40" · RPF = নিজের ঘরে ফেরার পথে এসেছে? · লিকি = ফুটো কল, টোকেন = সঞ্চয়ী হিসাব · X.25 প্রতি হপে যাচাই, FR কোথাও না, ATM কেটে 53 · অফসেট × 8 = বাইটের অবস্থান



নেটওয়ার্ক লেয়ারের প্রতিটি প্রশ্নপত্রের চারটি মূল প্রশ্ন

✓ অনুশীলনী MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. ডিসট্যান্স ভেক্টর রাউটিং কোন অ্যালগরিদমের ওপর ভিত্তি করে?

ক) Dijkstra খ) Prim
গ) Bellman-Ford ঘ) Kruskal

২. RIP-এ হপ কাউন্ট 16 মানে:

ক) অগম্য খ) সেরা রুট
গ) 16টি রাউটার ঘ) ডিফল্ট রুট

৩. OSPF সরাসরি IP-তে যে প্রোটোকল নম্বরে বহন হয়:

ক) 6 খ) 17
গ) 520 ঘ) 89

৪. কোন প্রোটোকল অটোনোমাস সিস্টেমগুলোর মধ্যে রাউটিং করে?

ক) BGP খ) RIP
গ) OSPF ঘ) IS-IS

৫. লিকি বাকেট অ্যালগরিদম মূলত দেয়:

ক) সংক্ষিপ্ততম পথ খ) স্থির আউটপুট রেট
গ) ত্রুটি শনাক্তকরণ ঘ) অ্যাড্রেস রেজল্যুশন

৬. টোকেন বাকেট: C = 250 KB, ρ = 2 MB/s, M = 25 MB/s। বার্স্টের দৈর্ঘ্য ≈

ক) 125 ms খ) 10 ms
গ) 11 ms ঘ) 23 ms

৭. কোন ডিভাইস ব্রডকাস্ট ডোমেইন ভাগ করে?

ক) হাব খ) রাউটার
গ) সুইচ ঘ) রিপিটার

৮. একটি ATM সেলের আকার:

ক) 48 বাইট খ) 64 বাইট
গ) 1500 বাইট ঘ) 53 বাইট

৯. 4000-বাইট ডেটাগ্রাম (20 B হেডার), MTU 1500: শেষ ফ্র্যাগমেন্টের অফসেট?

ক) 370 খ) 2960
গ) 185 ঘ) 3

১০. হপ-বাই-হপ এরর কন্ট্রোল কোনটির বৈশিষ্ট্য?

ক) Frame Relay খ) X.25
গ) ATM ঘ) MPLS

উত্তরমালা → ১-গ · ২-ক · ৩-ঘ · ৪-ক · ৫-খ · ৬-গ · ৭-খ · ৮-ঘ · ৯-ক · ১০-খ

লিখিত প্রশ্ন: উদাহরণসহ ডিসট্যান্স ভেক্টর রাউটিংয়ের কাউন্ট-টু-ইনফিনিটি সমস্যা ব্যাখ্যা করো এবং এটি কমানোর দুটি কৌশল বর্ণনা করো। ৮ নম্বর

১. DV রাউটার শুধু প্রতিবেশীর দূরত্ব জানে (Bellman-Ford), তাই প্রতিবেশীর রুট তার নিজের মধ্য দিয়েই ফিরে আসে কি না বুঝতে পারে না ১

২. শৃঙ্খল A–B–C, প্রতিটি কস্ট 1। B: A = 1; C: A = 2 (B হয়ে)। A–B লিংক বিচ্ছিন্ন হলো ১

৩. B শোনে C-এর "A = 2", তাই C হয়ে A = 3 ধরে; তারপর C ধরে B হয়ে A = 4; এভাবে 5, 6, … করে ∞ (RIP-এ 16) পর্যন্ত গোনে, পুরো সময় রাউটিং লুপ থাকে ৩

৪. **স্প্লিট হরাইজন**: যে লাইন থেকে শিখেছে সেই B-কে C আর A-র রুট ঘোষণা করে না; **পয়জন রিভার্স**: B-কে A = ∞ ঘোষণা করে ২

৫. আরও আছে হোল্ড-ডাউন টাইমার ও ট্রিগারড আপডেট; কোনোটিই ৩+ রাউটারের লুপ পুরোপুরি সারায় না, তাই লিংক স্টেট বেশি পছন্দনীয় ১

পরীক্ষার আগে ✓: ৫-নোডের গ্রাফে টেবিল করে Dijkstra · Bellman-Ford দিয়ে DV আপডেট · $S = C/(M - \rho)$ ও $C + \rho t$ · ফ্র্যাগমেন্টের আকার, অফসেট, MF · আরও লিখে অনুশীলন: DV বনাম LS, লিকি বনাম টোকেন বাকেট, X.25 বনাম FR বনাম ATM